Desafio Técnico — Pessoa Engenheira Full Stack / Platform (Pleno)

Bem-vinda(o) ao desafio técnico da Prefeitura do Rio. Ele existe para que possamos **conhecer como você pensa**, e não só o que você consegue entregar. Por isso, o que mais nos interessa é o caminho: as decisões que você tomou, o que você descartou, o que deu errado e como você investigou.

> **Sobre uso de IA:** você pode usar (assistentes de código, chat, o que quiser). Só pedimos duas coisas: **declare onde usou** e **seja capaz de explicar e defender cada linha** na entrevista. Entregar algo que você não consegue explicar pesa mais contra do que entregar algo incompleto.

---

1. Contexto

Imagine uma versão simplificada do fluxo central do Taxi.Rio: **passageiros solicitam corridas, motoristas aceitam, e o sistema acompanha o status em tempo quase real.**

Você vai construir (e operar) um pequeno serviço chamado `corridas` com este fluxo:



Componente	Responsabilidade
API	`POST /corridas` (cria), `PATCH /corridas/:id/status` (aceita/inicia/finaliza), `GET /corridas/:id` (lê, usando cache). Autenticação via JWT.
Worker	Consome eventos `corrida.criada` / `corrida.status_alterado`, atualiza o cache no Redis e grava histórico no MongoDB.
UI (React)	Tela mínima: login com token, criar corrida, listar/acompanhar status. Visual simples; **não vamos avaliar design**.
Broker	RabbitMQ.

Você pode usar NestJS ou Node puro (Express/Fastify). Se escolher NestJS, ótimo, mas não é obrigatório.

---

2. Como o desafio funciona

O desafio tem **3 partes**. A Parte 1 é a base; as Partes 2 e 3 são onde mais enxergamos seu raciocínio. **Você não precisa fazer tudo.** Esperamos que você priorize, e queremos saber o porquê da priorização.

Tempo sugerido: 6 a 8 horas, distribuídas em até 5 dias corridos. Não queremos que você gaste mais do que isso. Se estourar, pare e documente o que ficou de fora.

Parte 1 — Fundação (obrigatória)

1. **Ambiente local reproduzível** com `docker compose up` subindo API, worker, UI, MySQL, Redis, MongoDB, RabbitMQ e Prometheus.
2. **API e worker funcionando** conforme o fluxo acima, com:
   - validação de JWT (token estático pré-gerado + JWKS ou chave pública; explique a escolha). Cenários mínimos: sem token → 401, token inválido → 401, token válido → 2xx;
   - perfil no token: `passageiro` só cria/lê as próprias corridas; `motorista` só altera status. Demonstre o `403`;
   - **idempotência**: reenviar o mesmo `POST /corridas` com a mesma `Idempotency-Key` não cria corrida duplicada;
   - **tolerância a mensagem duplicada** no worker (entrega "at-least-once").
3. **Testes automatizados** que cubram ao menos o fluxo feliz e dois cenários de falha à sua escolha.
4. **Pipeline de CI no GitHub Actions** que rode lint, testes e build das imagens a cada push/PR.

Parte 2 — Operar (peso alto)

O que separa "funciona na minha máquina" de "dá pra operar em produção".

1. **Logs estruturados** (JSON) com um `correlation_id` / `trace_id` que atravesse API → broker → worker.
2. **Métricas** em formato Prometheus (endpoint `/metrics`) na API e no worker, incluindo ao menos: taxa de requisições, latência (histograma), erros e **tamanho/atraso da fila**.
3. **Um trace distribuído** (OpenTelemetry) mostrando uma corrida da API até o worker. Pode usar Jaeger/Tempo no compose.
4. **Dois alertas** com justificativa: *por que esses dois, e por que esses limiares?* Não precisa subir Alertmanager; a regra e o raciocínio bastam. Entregue as regras em dois arquivos com os mesmos `groups`:
   - `observability/alerts.yml`, no formato de regras do Prometheus, carregado por um Prometheus no `docker compose`, para que o avaliador veja os alertas em `http://localhost:9090/alerts`;
   - `observability/alerts-prometheusrule.yaml`, as mesmas regras como `PrometheusRule` (CRD do Prometheus Operator), formato que usamos no cluster.
5. **Teste de carga com k6** (script no repositório) e **um relatório curto** com: o cenário, o que você mediu, o que descobriu e o que mudaria a partir disso.

Parte 3 — Kubernetes (peso médio)

1. Manifestos (ou Helm chart/Kustomize) para subir a aplicação em um cluster **local** (`kind`, `k3d`, `minikube`, o que preferir).
2. `Deployment` com **probes** (liveness/readiness) e **resources** (requests/limits) coerentes, e uma explicação de como você chegou nesses valores.
3. `Secret` / `ConfigMap` para configuração, sem credenciais em texto no repositório.

4. **Bônus:** KEDA com scaler RabbitMQ escalando o worker pelo tamanho da fila, com demonstração durante o teste k6.

---

3. O que queremos ver no seu processo (obrigatório)

Esta é a parte mais importante. Entregue **além do código**:

3.1 `DECISIONS.md`

Um registro curto das **decisões relevantes** (3 a 8 entradas). Para cada uma: contexto, alternativas que considerou, o que escolheu e **o trade-off aceito**. Exemplos de perguntas que esperamos ver respondidas: *Que topologia usou no RabbitMQ (exchanges, filas, DLQ)? Cache-aside ou write-through? Onde garantir idempotência: banco, aplicação ou ambos? Por que MySQL/Mongo para cada dado?*

Uma das entradas é obrigatória: **se você fosse desenvolver esse sistema do zero, sem tecnologias definidas, manteria o RabbitMQ ou optaria por outro broker? Justifique sua resposta.**

3.2 `DIARIO.md`

Um diário de bordo **cronológico e informal**: o que você tentou, o que quebrou, como investigou, o que aprendeu. Erros e becos sem saída são bem-vindos; é aqui que você mostra seu método. Uma linha por ocorrência já ajuda.

3.3 Histórico de commits

Commits pequenos e frequentes, com mensagens que expliquem o *porquê*. Pedimos que **não faça squash** do histórico. Um único commit "initial" no fim do desafio dificulta nossa leitura do seu processo.

3.4 `LIMITES.md`

O que **não** fez, o que faria com mais tempo e quais riscos conhecidos ficaram abertos. Reconhecer limites é um ponto positivo.

3.5 Declaração de uso de IA (no `README` da sua entrega)

Em poucas linhas: quais ferramentas usou, para quê, e um exemplo de uma sugestão que você **rejeitou ou corrigiu** e por quê.

3.6 Vídeo de apresentação (até 10 min)

Um vídeo curto (Loom ou similar) em que você: (a) mostra o sistema rodando; (b) explica **uma decisão difícil** que tomou; (c) mostra **um problema que encontrou** e como o diagnosticou. Não precisa de edição nem de roteiro formal.

---

4. Regras e dicas

- **Prioridade:** uma Parte 1 sólida e bem explicada vale mais do que as três partes superficiais.
- **Não será avaliado:** design da UI, alta disponibilidade do cluster, serviços cloud pagos, otimização extrema de performance, cobertura de testes de 100%.
- **Segurança básica:** nada de segredos reais no repositório. Chaves e tokens de exemplo devem ser claramente de teste.
- **Dúvidas:** pode nos perguntar a qualquer momento; perguntar bem é um sinal positivo. Se algo estiver ambíguo, escolha uma interpretação razoável, registre no `DECISIONS.md` e siga em frente.

---

5. Entrega

Envie o link de um **repositório Git (público ou com acesso concedido)** contendo:

```
.
├── README.md              # como subir, testar e validar (comandos copiáveis) + declaraçã
├── DECISIONS.md
├── DIARIO.md
├── LIMITES.md
├── docker-compose.yml
├── api/  worker/  ui/
├── k8s/                   # manifestos, Helm ou Kustomize (Parte 3)
├── observability/         # config de Prometheus, alerts.yml + alerts-prometheusrule.yaml
├── loadtest/              # script k6 + relatório
└── .github/workflows/     # CI
```

O `README.md` deve permitir que um avaliador, com Docker instalado, **reproduza tudo seguindo apenas os comandos nele**. Vamos tentar rodar.

6. O que acontece depois

Quem avançar será convidado para uma **entrevista técnica de ~60 min**, em que vamos percorrer a sua solução em conjunto e **propor uma pequena evolução de requisito ao vivo**, a ser discutida (não precisa implementar tudo). Por isso, escolha o que você consegue explicar com segurança.

Boa sorte, e se divirta!